Institute for Thermal Turbomachinery
Prof. Dr.-Ing. H.-J. Bauer, Ord.

Analysis of Feature Importance in a Neural Network for Predicting the Remaining Useful Life of Aircraft Engines

Master's Thesis
by
Callum Dew, B.Sc.

Supervisor: **Joel Arweiler, M.Sc.**

April 2026

Karlsruher Institut für Technologie

Ich versichere, die Arbeit selbständig verfasst und nur die angegebenen Quellen und Hilfsmittel verwendet zu haben. Die wörtlich oder inhaltlich übernommenen Stellen sind als solche kenntlich gemacht. Die Satzung des Karlsruher Instituts für Technologie (KIT) zur Sicherung guter wissenschaftlicher Praxis wurde von mir in der aktuellen Fassung beachtet.

Karlsruhe, April 2, 2026

Contents

List of Symbols iii

List of Figures iv

List of Tables v

1 Introduction 1

2 Related Work 2
2.1 RUL Prediction using CMAPSS . . . 2
2.2 . . . 2
2.3 Research Gap and Thesis Positioning . . . 2

3 Fundamentals 3
3.1 Model-Agnostic Methods . . . 3
3.1.1 Individual Conditional Expectation . . . 3
3.1.2 Feature Ablation . . . 4
3.1.3 Shapley Additive Explanations . . . 5
3.1.4 Local Interpretable Model-Agnostic Explanations . . . 7
3.1.5 Counterfactual Explanations . . . 8
3.1.6 Permutation Feature Importance . . . 9
3.1.7 Accumulated Local Effects . . . 10
3.2 Model-Specific Methods . . . 12
3.2.1 Integrated Gradients . . . 12

4 Methodology 14
4.1 Dataset and Data Preparation . . . 14
4.1.1 Features set / sensors used . . . 14
4.1.2 Target definition (RUL label) . . . 15
4.2 Model Setup . . . 16
4.3 Training Procedure . . . 16
4.3.1 Single model setup . . . 17
4.3.2 Pairwise model setup . . . 18
4.3.3 All models system . . . 18
4.4 Implementation of Feature Importance Methods . . . 18
4.5 Unified Feature Importance Representation . . . 20

5 Results 22
5.1 Verification of FI algorithms . . . 22
5.2 Comparison: Single vs Combined Models . . . 22
5.3 Feature Importance Results per Method . . . 22
5.4 Comparison . . . 22

5.5 Discussion 22

6 Conclusion and Outlook **23**
6.1 Summary of Contributions 23
6.2 Answering the Research Questions 23
6.3 Outlook 23

References **24**

Appendix **26**
A.1 Anhang 1 26
A.2 Anhang 2 26

List of Symbols

Symbol	Unit	Description
Latin symbols		
F_{xy}		Faktor xy
H	m	Höhe
c_p	$\mathrm{J/(kg\,K)}$	Spezifische Wärmekapazität
Greek symbols		
κ		Isentropenexponent
λ	$\mathrm{W/(m\,K)}$	Wärmeleitfähigkeit
ε	$\mathrm{m^2/s^3}$	Dissipationsrate
Similarity parameters		
Bi		Biotzahl
Nu		Nußeltzahl
Pr		Prandtlzahl
Re		Reynoldszahl
Indices		
r		r-Koordinate
x		x-Koordinate
y		y-Koordinate
Abbreviations		
CFD		Computational Fluid Dynamics
FEM		Finite-Elemente-Methode

List of Figures

4.1 Structure of the input tensor for a single sliding window sample, consisting of 13 sensor channels over 70 consecutive time steps. Each slice in the depth direction represents one engine unit. 15
4.2 Architecture of the 1D convolutional neural network. The red window illustrates the receptive field of the convolutional kernel ($k = 5$). Green: feature extractor, blue: label predictor. 17

List of Tables

4.1 Sensor features retained after non-informative sensor removal. 14

A.1 Simulationsergebnisse, die den Lesefluss stören. 26

1 Introduction

The aviation industry is one of the most safety-critical and technologically sophisticated fields in the world. Ensuring the reliability and availability of aircraft is essential not only for passenger safety but also for the economic efficiency of airlines and engine manufacturers. As engines operate under extreme mechanical and thermal conditions, their degradation processes must be continuously monitored and understood. In recent years, predictive maintenance has become a central concept in aviation, aiming to anticipate component failures before they occur and to enable maintenance actions precisely when needed.

A key element of predictive maintenance is the estimation of the Remaining Useful Life (RUL) of engine components. RUL predictions allow operators to plan maintenance more effectively, reduce unscheduled downtime, and avoid unnecessary part replacements. However, obtaining accurate RUL predictions is challenging. The underlying degradation mechanisms are highly nonlinear and influenced by a large number of operational parameters. Traditional modelling techniques often struggle to capture this complexity.

With the rise of large sensor datasets and computational power, machine learning and deep learning methods have become increasingly important in RUL estimation. Neural networks, in particular, are capable of modelling complex relationships in engine data and have demonstrated strong predictive performance. However, these models introduce a new challenge: despite their accuracy, they often behave as black boxes, offering little insight into how individual features contribute to the prediction. In a safety-critical environment such as aviation, this lack of transparency poses a significant limitation. Understanding why a model arrives at a certain prediction is essential for trust, certification, and operational decision-making.

This challenge has led to the emergence of Explainable Artificial Intelligence (XAI). XAI methods aim to make complex AI systems more interpretable by providing insights into which input feature influence a model's output and to what extent. One of the most widely used approaches within XAI is the analysis of feature importance. Feature importance methods allow engineers and researchers to understand which parameters are most relevant for RUL estimation and how different models may focus on different aspects of the data.

2 Related Work

2.1 RUL Prediction using CMAPSS

Discuss prior CMAPSS work

2.2

Discuss prior interpretability work regarding RUL prediction

2.3 Research Gap and Thesis Positioning

prior work does NOT compare single vs combined models

limited comparison of feature importance redistribution

limited analysis of RUL in DANN Architectures

3 Fundamentals

This chapter introduces the theoretical foundations of feature importance methods for explaining machine learning models. It presents formal definitions, mathematical formulations, and key properties of widely used explanation techniques. The methods are structured according to how explainability methods can broadly be categorized. The two primary dimensions are **global vs. local explanations** and **model-agnostic vs. model-specific methods**. Applications-specific details are addressed in subsequent chapters.

3.1 Model-Agnostic Methods

Model-agnostic explanation methods operate independently of the underlying model structure. They require only the ability to query the model with input data and observe the resulting prediction. Formally, let

$$f : \mathbb{R}^d \to \mathbb{R} \tag{3.1}$$

be a trained predictive model that maps an input vector

$$x = (x_1, x_2, ..., x_d) \tag{3.2}$$

to a scalar output, such as a predicted RUL.

Model-agnostic methods aim to quantify the influence of each features x_i on the prediction $f(x)$ without access to the internal structure of f. Instead, they analyze how the model output changes when input features are perturbed, fixed, permuted, or replaced.

These methods can provide both global and local explanations depending on whether they evaluate effects across the dataset or at the level of individual samples. Their general applicability makes them particularly useful when comparing different models, as is the case in this thesis.

The following sections introduce several widely used model-agnostic explanation methods.

3.1.1 Individual Conditional Expectation

Individual Conditional Expectation (ICE) is a local explanation method that visualizes how the prediction of a model changes as a single feature varies, while all other features remain fixed at their observed (Goldstein et al., 2015). ICE can be viewed as a fine-grained extension of Partial Dependence Plots (PDP), providing instance-level insight rather than averaged effects.

Let $f(x)$ be a trained model and let

$$\mathbf{x} = (x_1^{(i)}, x_2^{(i)}, ..., x_d^{(i)}) \tag{3.3}$$

be a specific instance from the dataset. To analyze the effect of feature x_j, ICE constructs a function

$$ICE_j^{(i)}(z) = f(x_1^{(i)}, ..., x_{j-1}^{(i)}, z, x_{j+1}^{(i)}, ..., x_d^{(i)}) \tag{3.4}$$

where the value of feature x_j is varied over a range of interest z, while all other feature remain fixed.

This produces a curve representing the prediction as a function of feature x_j for a single instance. By computing such curves for multiple instances, ICE reveals how the relationship between a feature and the prediction varies across different regions of the input space.

Unlike Partial Dependence Plots, which average over all instances (global)

$$PDP_j(z) = \frac{1}{N}\sum_{i=1}^{N} ICE_j^{(i)}(z) \tag{3.5}$$

ICE preserves instance-level heterogeneity. This allows identification of feature interactions and non-linear effects that may be obscured by averaging.

ICE provides several important insights. For one, the local sensitivity shows how sensitive the model prediction is to changes in a specific feature for an individual instance. Heterogeneous effects can be derived in different ICE curves, which may show different slopes, indicating interactions between features. Non-linear dependencies between feature and predictions are also directly revealed by ICE curves.

However, ICE also has limitations. Since it evaluates feature values independently of their joint distribution, it may produce unrealistic feature combinations, especially in highly correlated datasets. Additionally, visualization becomes difficult when the number of instances or features is large.

Despite these limitations, ICE is a valuable tool for understanding local model behavior and serves as a foundation for related methods such as Partial Dependence Plots and Accumulated Local Effects.

In the context of this thesis, ICE is used as a local sensitivity analysis method. Feature importance can be derived by quantifying the magnitude of prediction changes across the feature range and aggregating these effects into comparable importance scores (more in chapter 4).

3.1.2 Feature Ablation

Feature Ablation is a model-agnostic explanation method that quantifies feature importance by measuring the change in the model prediction when one or more input features are removed or replaced. The underlying assumption is that a feature is important if its absence leads to a significant change in the model output (Zeiler and Fergus, 2014).

Let

$$f : \mathbb{R}^d \to \mathbb{R} \tag{3.6}$$

be a trained predictive model and let

$$\mathbf{x} = (x_1, ..., x_d) \tag{3.7}$$

denote a specific input vector. To evaluate the importance of feature x_j an ablated input $\mathbf{x}_{-j}$ is constructed in which the original value x_j is replaced by a reference value x_j^{ref}. This reference may correspond to the feature mean, zero, a baseline sample, or another domain-specific neutral value.

The local attribution for feature j is then defined as

$$\phi_j(\mathbf{x}) = f(\mathbf{x}) - f(\mathbf{x}_{-j}) \tag{3.8}$$

A large absolute of $\phi_j(\mathbf{x})$ indicates that the feature has a strong influence on the prediction for the considered instance.

Feature ablation naturally extends to groups of features. In structured inputs such as multivariate time-series data, entire sensors, time windows, or feature subsets can be ablated jointly. This enables interpretation at higher semantic levels and aligns with engineering-oriented analyses.

Global feature importance can be obtained by aggregating local attributions over a dataset $\{\mathbf{x}^{(i)}\}_{i=1}^{N}$:

$$FI_j = \frac{1}{N}\sum_{i=1}^{N} |\phi_j(\mathbf{x}^{(i)}| \tag{3.9}$$

Although conceptually simple and broadly applicable, feature ablation has several limitations. The resulting attributions depend strongly on the choice of reference values. Furthermore, replacing individual features independently may lead to unrealistic input configurations when features exhibit statistical dependencies, which can bias importance estimates (Molnar, 2022).

3.1.3 Shapley Additive Explanations

SHAP (SHapley Additive exPlanations) is a model-agnostic framework for attributing a model prediction to its input features using concepts from cooperative game theory (Lundberg and Lee, 2017, Shapley, 1953). SHAP assigns each feature and attribution value that represents its average marginal contribution to the prediction across all possible feature coalitions

Let a trained model be

$$\mathbf{f} : \mathbb{R}^d \to \mathbb{R}, \tag{3.10}$$

and let $\mathbf{x} = (x_1, ..., x_d)$ be an instance to be explained. In game-theoretic terms, each feature index $j \in \{1, ..., d\}$ is treated as a "player" in a coalition game. The central object is a **value function**

$$v(S) \in \mathbb{R}, S \subseteq \{1, ..., d\}, \tag{3.11}$$

which represents the model output when only the features in subset S are "present" (how "presence" is defined is discussed below). The Shapley value for feature j is then

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(d-|S|-1)!}{d!}(v(S \cup \{j\}) - v(S)), \tag{3.12}$$

where $F = \{1, ..., d\}$. The term $v(S \cup \{j\}) - v(S)$ is the **marginal contribution** of feature j to coalition S, and the factorial weight averages this contribution over all coalition sizes and orders (Shapley, 1953).

A common and useful special case is to explain the deviation from a baseline (expected) prediction:

$$f(\mathbf{x}) = \phi_0 + \sum_{j=1}^{d} \phi_j, \tag{3.13}$$

where ϕ_0 is a reference value typically chosen as $\mathbb{E}[f(\mathbf{X})]$ under a background distribution (Lundberg and Lee, 2017). This yields an additive decomposition of the prediction into a baseline plus feature contributions.

Shapley values are uniquely characterized by a set of axioms in cooperative game theory. In the context of feature attribution, the most commonly emphasized properties are the following:

- **Efficiency (additivity/completeness)**: attributions sum to the explained quantity, e.g. $f(\mathbf{x}) - \phi_0 = \sum_j \phi_0$.
- **Symmetry**: if two features contribute identically to all coailitions, they receive equal attribution.
- **Dummy (null player)**: if a feature never changes $v(S)$ when added its attribution is zero.
- **linearity**: for a linear combination of models, attributions combine linearly.

These properties motivate SHAP as a principled approach for local feature attribution.

To compute Shapley values for a predictive mode, the value function $v(S)$ must be defined in terms of the model. A common definition is based on conditional expectations,

$$v(S) = \mathbb{E}[f(\mathbf{X}) | \mathbf{X}_S = \mathbf{x}_S] \tag{3.14}$$

which keeps the features in S fixed to their observed values and averages over the remaining features according to the conditional distribution (Aas et al., 2021, Štrumbelj and Kononenko, 2014). This formulation attempts to respect statistical dependencies between features but requires estimation of potentially high-dimensional conditional distributions. An alternative interventional formulation defines

$$v(S) = \mathbb{E}[f(\mathbf{x}_S, \mathbf{X}_{\overline{S}})], \tag{3.15}$$

where the complementary features $\mathbf{X}_{\overline{S}}$ are drawn independently from a background distribution (Aas et al., 2021). While computationally more tractable, this approach may evaluate the model on feature combinations that are unlikely under the true data distribution. In the presence of correlated features, different definitions of $v(S)$ can lead to different attributions (Aas et al., 2021).

Lundberg and Lee (2017) further show that SHAP belongs to the class of additive explanation models of the form

$$g(\mathbf{z}') = \phi_0 + \sum_{j=1}^{d} \phi_j z'_j, \tag{3.16}$$

where $\mathbf{z}' \in \{0,1\}^d$ is a binary vector indicating feature presence. Under appropriate weighting of coalition samples and a specific loss function, the solution of a local surrogate regression problem recovers the Shapley values, thereby connecting SHAP to surrogate-based explanation approaches while retaining the axiomatic guarantees of cooperative game theory.

Exact computation of Shapley values requires evaluation of all 2^d feature coalitions and is therefore computationally intractable for high-dimensional models. Practical implementations rely on approximation strategies, including sampling-based estimation of marginal contributions (Štrumbelj and Kononenko, 2014), the KernelSHAP algorithm based on weighted linear regression (Lundberg and Lee, 2017), and efficient algorithms for specific model classes such as tree ensembles (Lundberg et al., 2019). Although these methods approximate the exact Shapley values, they aim to preserve the theoretical properties of the original formulation.

Despite its strong theoretical foundation, SHAP has limitations. Attributions depend on the choice of background distribution and on whether conditional or interventional semantics are used. In the presence of correlated features, Shapley values distribute shared predictive information according to the chosen value function, which does not necessarily correspond to causal influence (Aas et al., 2021). Moreover, approximation methods can introduce variance and computational overhead in high-dimensional settings. Nevertheless, SHAP remains one of the most widely adopted feature attribution frameworks due to its axiomatic grounding, additive structure, and general applicability.

3.1.4 Local Interpretable Model-Agnostic Explanations

Local Interpretable Model-Agnostic Explanations (LIME) is a model-agnostic method that explains individual prediction by approximating the original model locally with an interpretable surrogate model (Ribeiro et al., 2016). The central premise is that, although a complex model may exhibit highly non-linear global behavior, its prediction surface can often be approximated by a simpler model in a sufficiently small neighborhood around a specific input instance.

Let

$$f : \mathbb{R}^d \to \mathbb{R} \tag{3.17}$$

be a trained predictive model and let $\mathbf{x} \in \mathbb{R}^d$ denote the instance to be explained. LIME generates a set of perturbed samples $\{\mathbf{z}_k\}_{k=1}^K$ in the vicinity of $\mathbf{x}$, together with corresponding predictions $f(\mathbf{z}_k)$. A locality kernel $\pi_x(\mathbf{z})$ is introduced to assign weights to perturbed samples based on their proximity to $\mathbf{x}$, typically decreasing with increasing distance.

An interpretable surrogate model g is then fitted from a restricted hypothesis class G, such as sparse linear models, by solving the optimization problem

$$\arg\min_{g \in G} \mathscr{L}(f, g, \pi_x) + \Omega(g) \tag{3.18}$$

where $\mathscr{L}(f, g, \pi_x)$ is a locality-weighted loss function measuring how well g approximates f in the neighborhood of $\mathbf{x}$, and $\Omega(g)$ is a regularization term that penalizes model complexity to enforce interpretability (Ribeiro et al., 2016).

When g is chosen as a linear model,

$$g(\mathbf{z}) = w_0 + \sum_{j=1}^{d} w_j z_j, \tag{3.19}$$

the coefficients w_j serve as local feature importance scores for the prediction at $\mathbf{x}$. These coefficients quantify how variations in individual features influence the model output within the defined local neighborhood.

LIME provides local explanations by construction and does not impose constrains on the global behavior of the original model. The resulting explanation depends on several design choices, including the sampling distribution for perturbed instances, the definition of the proximity kernel, and the regularization applied to the surrogate model. Consequently, explanations may vary across different runs due to stochastic perturbation sampling.

Despite these limitations, LIME offers a flexible and intuitive framework for local model explanation by explicitly separating the complex predictive model from the interpretable surrogate used for explanation.

3.1.5 Counterfactual Explanations

Counterfactual explanations provide local explanations by identifying minimal changes to an input instance that would alter the model's prediction to a desired outcome (Wachter et al., 2018). Instead of attributing importance scores to individual features, counterfactual methods characterize the decision boundary of a model by answering the question: *What is the smallest modification to the input that would change the prediction?* As such, counterfactual explanations are classified as a local method and focus on the behavior of the model in the vicinity of a specific instance (Wachter et al., 2018).

Let

$$f : \mathbb{R}^d \to \mathbb{R} \tag{3.20}$$

be a predictive model and let $\mathbf{x} \in \mathbb{R}^d$ denote the instance under consideration. For a desired target outcome y', a counterfactual explanation seeks a modified input $\mathbf{x}^{cf}$ that satisfies

$$f(\mathbf{x}^{cf}) = y'. \tag{3.21}$$

Among all such inputs, the preferred counterfactual is typically defined as the one that minimizes a distance measure $D(\mathbf{x}, \mathbf{x}^{cf})$, which quantifies the magnitude of change between the original and modified instance. Formally, the counterfactual problem can be expressed as

$$\mathbf{x}^{cf} = \arg\min_{\mathbf{z}} D(\mathbf{z}, \mathbf{x}) \quad \text{subject to} \quad f(\mathbf{z}) = y'. \tag{3.22}$$

In practice, this constrained optimization problem is often relaxed into an unconstrained formulation,

$$\mathbf{x}^{cf} = \arg\min_{\mathbf{z}} \lambda \cdot \mathscr{L}(f(\mathbf{z}), y') + D(\mathbf{z}, \mathbf{x}), \tag{3.23}$$

where $\mathscr{L}$ is a loss function that penalizes deviations from the desired prediction y', and $\lambda > 0$ balances prediction fidelity and proximity to the original input (Wachter et al., 2018). The distance function D is commonly chosen as an L_1, L_2, or weighted norm, although other metrics can be employed to reflect domain-specific constraints.

Counterfactual explanations are closely related to optimization and adversarial perturbation techniques, but differ in their interpretative objective. Adversarial examples aim to expose model vulnerabilities, while counterfactual explanations aim to provide meaningful and interpretable insights into the model's decision boundary, therefore the feature importance. Specifically, they identify directions in the input space that lead to a change in prediction, thereby revealing which features must vary to alter the model's output.

Several extensions to the basic formulation have been proposed. For example, constraints can be imposed to ensure plausibility, feasibility, or sparsity of the counterfactual instance (Karimi et al., 2020, Verma et al., 2022). Additionally, causal considerations may be incorporated to ensure that suggested changes respect structural dependencies between features (Kusner et al., 2018). Such constraints are particularly important when features are correlated or when certain attributes are immutable.

Despite their intuitive appeal, counterfactual explanations present several challenges. The optimization problem may be non-convex for complex models, leading to multiple local minima. The resulting explanations depend strongly on the choice of distance metric, regularization parameters, and imposed constraints. Moreover, multiple valid counterfactuals may exist for a single instance, and there is generally no unique solution.

Nevertheless, counterfactual explanations provide a powerful local perspective on model behavior by directly characterizing the geometry of decision boundaries. Rather than distributing credit across features, they describe actionable changes in the input space that would alter the model's prediction, thereby offering a complementary viewpoint to attribution-based explanation methods.

3.1.6 Permutation Feature Importance

Permutation Feature Importance (PFI) is a model-agnostic method that quantifies the global importance of input features by measuring the degradation in predictive performance when the association between a feature and the target variable is disrupted (Breiman, 2001, Fisher et al., 2019). The underlying idea is that if a feature is important for the model's predictions, then randomly permuting its values across the dataset should lead to a measurable decline in model performance.

Let

$$f : \mathbb{R}^d \to \mathbb{R} \tag{3.24}$$

be a trained predictive model and let

$$\{(\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^{N} \tag{3.25}$$

be a dataset of input-output pairs. Let $\mathscr{L}(f)$ denote a chosen performance metric, such as mean squared error or classification accuracy, evaluated on the dataset. For a given feature index $j \in \{1,...,d\}$, a permuted dataset is defined by randomly shuffling the values of feature x_j across all samples while leaving all other features unchanged. Denote the resulting modified dataset by $\tilde{\mathbf{x}}^i_{(j)}$.

The permutation importance of features j is then defined as

$$FI_j = \mathscr{L}(f(\tilde{\mathbf{x}}_j)) - \mathscr{L}(f(\mathbf{x})), \tag{3.26}$$

where the first term represents the performance of the model evaluated on the dataset with feature j permuted, and the second term denotes the performance on the original dataset. For err-based metrics, a larger value of FI_j indicates greater importance.

Rather than measuring a causal effect of the feature, PFI accounts for the predictive reliance of the model. By permuting a feature independently of the others, the method removes the information contained in that feature while preserving the marginal distributions of the remaining variables. Since the procedure operates entirely on input-output behavior, it is applicable to any predictive model without requiring access to internal parameters.

Although conceptually simple, permutation importance has several limitations. When features are correlated, permuting one feature may create unrealistic input combinations, which can lead to inflated or misleading importance estimates (Fisher et al., 2019). In these cases, the measured importance may reflect the extent to which a feature shares information with correlated variables rather than its unique contribution. Furthermore, the resulting importance values depend on the choice of performance metric and the evaluation dataset.

Despite these limitations, PFI remains a widely used global explanation method due to its simplicity, general applicability, and direct interpretation in terms of performance degradation.

3.1.7 Accumulated Local Effects

Accumulated Local Effects (ALE) is a model-agnostic global method that estimates the average influence of a feature on model predictions while accounting for feature correlations (Apley and Zhu, 2019). ALE was introduced as an alternative to PDP, which may produce misleading interpretations when features are statistically dependent. Unlike PDP, ALE estimates feature effects locally and accumulates them over the empirical distribution of the data.

Let

$$f : \mathbb{R}^d \to \mathbb{R} \tag{3.27}$$

be a trained predictive model and let

$$\mathbf{x} = (x_1, ..., x_d) \tag{3.28}$$

denote an input vector drawn from the joint distribution of features. To quantify the effect of feature x_j ALE partitions the domain of x_j into K disjoint intervals defined by grid points

$$z_0 < z_1 < \cdots < z_K, \tag{3.29}$$

typically chosen according to empirical quantiles of the observed data.

For each interval $[z_{k-1}, z_k]$, ALE computes the average local change in prediction when feature x_j moves from the lower to the upper boundary of the interval, while all other features remain fixed. For samples satisfying $x_j^{(i)} \in [z_{k-1}, z_k]$, the local effect is defined as

$$\Delta_{j,k}^{(i)} = f(x_1^{(i)}, ..., z_k, ..., x_d^{(i)} - f(x_j^{(i)}, ..., z_{k-1}, ..., x_d^{(i)}). \tag{3.30}$$

The average local effect within the interval is then computed as

$$\hat{\Delta}_{j,k} = \frac{1}{N_k} \sum_{i:x_j^{(i)} \in [z_{k-1}, z_k]} \Delta_{j,k}^{(i)}, \tag{3.31}$$

where N_k denotes the number of samples in the interval.

The accumulated local effect up to a point z in the domain of feature x_j is obtained by summing the average local effects of all intervals with upper bound less than or equal to z:

$$ALE_j(z) = \sum_{k:z_k \leq z} \hat{\Delta}_{j,k}. \tag{3.32}$$

To ensure interpretability, ALE functions are typically centered such that their mean over the empirical distribution of x_j is zero:

$$\int ALE_j(z) dP_{X_j}(z) = 0. \tag{3.33}$$

In the limit of infinitesimally small intervals, ALE can be interpreted as an accumulated conditional expectation of partial derivatives,

$$ALE_j(z) = \int_{z_0}^{z} \mathbb{E}\left[\frac{\partial f(\mathbf{X})}{\partial x_j} \middle| X_j = t\right] dt, \tag{3.34}$$

which highlights its local character (Apley and Zhu, 2019).

A key distinction between ALE and PDP lies in the condition in the conditioning mechanism. PDP evaluates

$$\mathbb{E}[f(x_j, \mathbf{X}_{-j})], \tag{3.35}$$

thereby averaging prediction over the marginal distribution of the remaining features. When features are correlated, this may result in evaluations at unrealistic feature combinations. In contrast, ALE computes differences within local intervals conditioned on observed data, thus respecting the empirical joint distribution of features and reducing extrapolation beyond the data support.

ALE provides a global explanation in the form of a function $ALE_j(z)$, which describes how the prediction changes as the feature x_j varies in its domain. However, ALE does not decompose individual prediction into additive feature contributions. Instead, it characterizes the average effect of a feature over the dataset.

Despite its advantages over PDP in the presence of correlated variables, ALE remains sensitive to the choice of binning strategy and the density of data within intervals. Sparse regions of the feature space may lead to unstable estimates of local differences. Nevertheless, ALE offers a principled and computationally efficient approach for estimating global feature effects while mitigating the extrapolation issues inherent in marginal averaging methods.

3.2 Model-Specific Methods

Model-specific explanation methods exploit the structural information about the underlying predictive model and make use of internal quantities such as gradients, activations, or parameter values. In contrast to model-agnostic approaches, which rely solely on input-output behavior. As a result, they are typically restricted to particular model classes, most commonly differentiable models such as neural networks.

Model-specific methods are frequently designed to provide local explanations by attributing a single prediction to its input features. However, global insights can be obtained by aggregating local attributions across multiple samples. Compared to model-agnostic techniques, these methods often offer improved computational efficiency and stronger theoretical guarantees within their applicable model class, but at the cost of reduced generality.

The following section introduces a gradient-based attribution method that is widely used for explaining differentiable models.

3.2.1 Integrated Gradients

Integrated Gradients (IG) is a model-specific, gradient-based attribution method for explaining predictions of differentiable models (Sundararajan et al., 2017). The method assigns feature attributions by integrating the gradients of the model output with respect to the input features along a continuous path from a baseline input to the instance being explained. Unlike simple gradient-based saliency methods, IG mitigates issues related to gradient saturation and local non-linearity by accumulating gradients over the entire path.

Let

$$f : \mathbb{R}^d \to \mathbb{R} \tag{3.36}$$

be a differentiable predictive model and let $\mathbf{x} \in \mathbb{R}$ denote the input instance of interest. Furthermore, let $\mathbf{x}' \in \mathbb{R}^d$ be a predefined baseline input representing the absence of features or a reference state. The baseline plays a central role in the attribution process and defines the point from which feature contributions are accumulated.

Integrated Gradients constructs a straight-line path between the baseline and the input:

$$\gamma(\alpha) = \mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}'), \quad \alpha \in [0,1]. \tag{3.37}$$

The attribution assigned to feature j is defined as

$$\phi_i(\mathbf{x}) = (x_j - x'_j) \int_0^1 \frac{\partial f(\gamma(\alpha))}{\partial x_j} d\alpha. \tag{3.38}$$

this formulation can be interpreted as the path integral of the gradient component corresponding to feature j, scaled by the difference between the input and the baseline value of that feature. Intuitively, it measures the accumulated sensitivity of the model output to changes in x_j along the path from $\mathbf{x}'$ to $\mathbf{x}$.

A central theoretical property of IG is **completeness**, also referred to as summation to delta (Sundararajan et al., 2017):

$$\sum_{i=1}^{d} \phi_j(\mathbf{x}) = f(\mathbf{x}) - f(\mathbf{x}'). \tag{3.39}$$

This phenomenon ensures that the total attribution equals the difference in model output between the input and the baseline, providing an additive decomposition of the prediction difference.

IG was proposed to satisfy two axioms that simple gradient methods may violate: **sensitivity** and **implementation invariance** (Sundararajan et al., 2017). Sensitivity requires that if two inputs differ in only one feature and yield different predictions, the differing feature must receive a non-zero attribution. Implementation invariance states that functionally equivalent models (models representing the same function but implemented differently) should produce identical attributions. Standard gradient methods may fail to satisfy these properties, whereas IG provably satisfies them under mild regularity conditions.

In practice, the integral is approximated using a Riemann sum with m steps:

$$\phi_j(\mathbf{x}) \approx (x_j - x'_j) \frac{1}{m} \sum_{k=1}^{m} \frac{\partial f(\gamma(\alpha_k))}{\partial x_j}, \tag{3.40}$$

where $\alpha_k = \frac{k}{m}$. Increasing m improves the approximation accuracy but increases computational cost.

Despite its theoretical appeal, IG has several limitations. The resulting attributions depend on the choice of baseline $\mathbf{x}'$, and there is generally no canonical baseline for many applications. Different baselines may yield different attributions values. Furthermore, the straight-line path assumption may not reflect meaningful trajectories in input space, particularly when features are highly structured or constrained. Extensions such as path-integrated gradients over alternative paths have been proposed to address some of these concerns (Sundararajan et al., 2017).

IG provides local explanations by attributing a single prediction to its input features. Global feature importance can be obtained by aggregating attributions across multiple samples, but the method itself is fundamentally local and defined at the level of individual inputs.

4 Methodology

This chapter describes the experimental setup used to analyse feature importance in neural network-based RUL prediction. It covers the dataset and preprocessing pipielin, the model arcchitecture and its three configurations, the training procedure, and the implementation of feature importance methods. The models themselves are not the primary contribution of this thesis; they serve as controlled predictive systems under which feature importance behaviour can be systematically studied across different input configurations.

4.1 Dataset and Data Preparation

4.1.1 Features set / sensors used

The raw input space of each CMAPSS sub-dataset consitst of three operational settings and 21 sensor measurements per time step. Eight sensors: s1, s5, s6, s10, s14, s16, s18, and s19, were identified as non-informative due to near-constant values across all trajectories and were excluded from the analysis. The remaining 13 sensor channels, listed in 4.1, form the feature set used throughout this work.

Table 4.1: Sensor features retained after non-informative sensor removal.

Senso ID	Physical Description
s2	LPC outlet temperature
s3	HPC outlet temperature
s4	LPT outlet temperature
s7	HPC outlet pressure
s8	Fan speed
s9	Core speed
s11	HPC outlet static pressure
s12	Ratio to Fuel flow to Ps30
s13	Corrected fan speed
s15	Bypass ratio
s17	Bleed enthalpy
s20	HPT cool air flow
s21	LPT cool air flow

Each sensor channel is scaled indepentently to the range [0, 1] using a MinMax scaler. Because sensosr readings vary systematically with ooperating condition, scaling is applied separately within each of the distinct operating regimes present in the data. The scaler is fitted exclusively on the training set.

Input sequences are constructed using a sliding window of length $T = 70$ time steps and stride 1, yielding tensors of shape (N x 30 x 70), where N is the number of sequences. The label

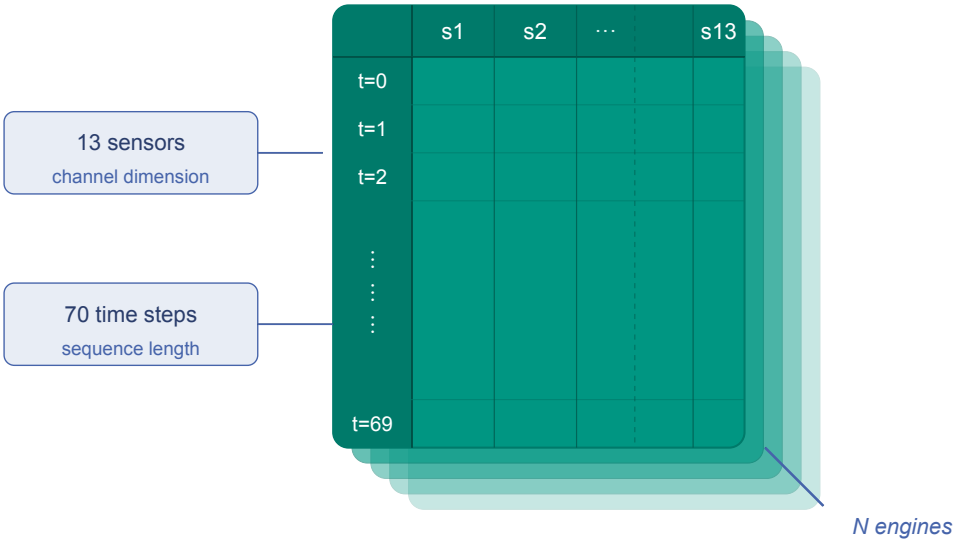

Figure 4.1: Structure of the input tensor for a single sliding window sample, consisting of 13 sensor channels over 70 consecutive time steps. Each slice in the depth direction represents one engine unit.

associated with each window is the RUL value at the final time step of that window. The resulting input strcture is illustrated in Figure 4.1.

4.1.2 Target definition (RUL label)

The raw RUL for each engine trajectory is computed as the difference between the end-of-life cycle t_{EOL} and the current cycle t:

$$\text{RUL}_{\text{raw}}(t) = t_{\text{EOL}} - t \tag{4.1}$$

A purely linear RUL label is known to dominate model training with the healthy regime, where no meaningful degradation signal is present. To address this, a piecewise linear transformatino is applied using a maximum RUL threshold $R_c = 125$ cycles. Early-life cycles, where the tru RUL exceeds R_c, are assigned a constant label of R_c, under the assumptioin that the engine shows no detectable degradation in this regime. Only once the trajectory enters the degradation region does the label decrease linearly toward zero at failure:

$$\text{RUL}(t) = min(\text{RUL}_{\text{raw}}(t), R_c). \tag{4.2}$$

This transformation prevents the model from being overwhelmed by healthy-state samples and focuses learning on the degradation region, which carries the operationally relevant signal.

Following the piecewise transfromation, RUL labels are normalized to [-1, 1] using a min-max scaling based on the training set:

$$\hat{y} = 2 * \frac{\mathrm{RUL}(t) - \mathrm{RUL}_{\min}}{\mathrm{RUL}_{\max} - \mathrm{RUL}_{\min}} - 1, \tag{4.3}$$

where $\mathrm{RUL}_{\min}$ and $\mathrm{RUL}_{\max} = R_c = 125$ by construction, so that a healthy engine at the cap maps to +1 and an engine at failure maps to -1. The same scaling parameters are applied to the test set without re-fitting.

4.2 Model Setup

All model variants used in this thesis share a common architecture base on a Domain-Adversarial Neural Netwrok (DANN) . The DANN consists of three components: a convolutional feature extractor, a label predictor, and an optional domain classifier connected via a Gradient Reversal Layer (GRL). Since the primary purpose of the models here is to serve as a stable substrate for feature importance analysis rather than to maximise transfer performance, the domain classifier branch is used selectively and the architecture is held fixed across all configurations.

The **Feature Extractor** processes the input tensor of shape $(Nx13xT)$ through four successive one-dimensional convolutional layers. The number of filters follows a halving schedule: 128, 64, 32, and 16 channels respectively, each with kernel size 5 and same-length padding of 2. After each convolutional layer, a LeakyReLu activation with negative slope 0.1 is applied. The temporal dimensions is collapsed using a global average pooling operation (AdaptiveAvgPool1d), which produces a fixed-length representation of 16 values regardless of input sequence length. A final fully connected layer projects this to a feature vector of length 64. Note that the slab height in Figure 4.2 is a visual convention representing the reducing channell count and does not imply a reduction in the temporal dimesnion, which remains 70 throughout.

The **Label Predictor** maps the 64-dimensioinal feature vector to a scalar RUL estimate through two fully connected layers: Linear $(64 \to 32)$ and Linear $(32 \to 1)$, each preceded by a ReLU activation. The output is a single value representing the predicted RUL.

4.3 Training Procedure

All models are trained to minimise the mean squared error (MSE) between the predicted and true normalised RUL values:

$$\mathscr{L} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}^{(i)} - y^{(i)})^2. \tag{4.4}$$

The models are trained with the following subset combinations.

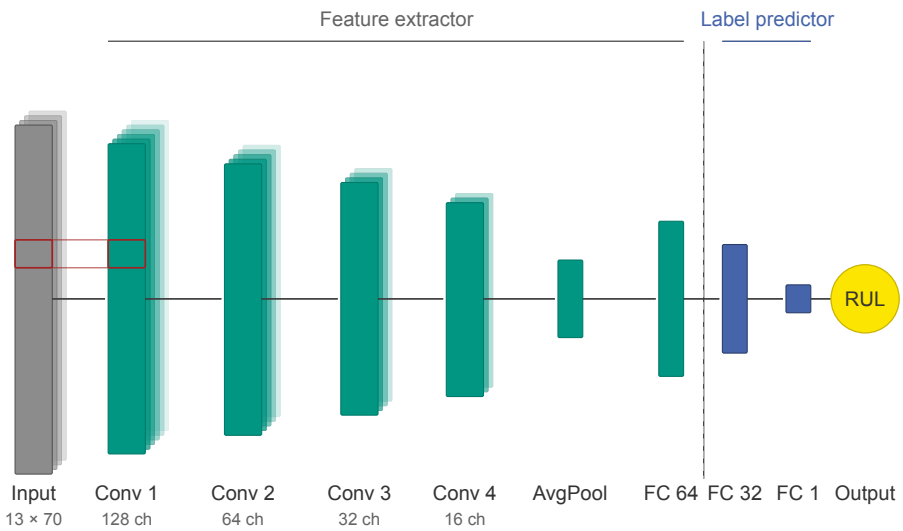

Figure 4.2: Architecture of the 1D convolutional neural network. The red window illustrates the receptive field of the convolutional kernel ($k = 5$). Green: feature extractor, blue: label predictor.

4.3.1 Single model setup

Four single-dataset models are trained, one for each CMAPSS sub-dataset (FD001, FD002, FD003, and FD004). Each model sees only the data from its respective sub-datasest during training. These models allow feature importance to be analysed individually per subb-datset,

revealing whether different operating conditions and fault moded lead to systematically different feature reliance.

4.3.2 Pairwise model setup

Six pairwise models are trained, one for each combination of two CMAPSS sub-dataset: FD001+FD002, FD001+FD003, FD001+FD004, FD002+FD003, FD002+FD004, and FD003+FD004. Training data from both sub-datasets are concatenated and used jointly. the motivation for this setup is to examine how feature importance redistributes when a model must reconcile two distinct operating regimes simultaneously, without yet being exposed to the full diversity of all four sub-datasets.

4.3.3 All models system

A single model is trained on the combined data from all four CMAPSS sub-satasets. This configuration represents the most demanding generalistaion scenario and serves as the counterpart to the single-dataset models, allowing a direct comparison of feature importance under narrow versus broad training distributions.

4.4 Implementation of Feature Importance Methods

All eight feature importance methods are applied post-hoc to the trained models, without any modification of the learned weights. A central implementation challenge shared across all methods is the multivariate time-series structure of the input: each sample is a tensor of shape (N x 13 x 70), where the 13 sensor channels and 70 time steps together constitute the model's input spcae. Most standard feature importance implementations assume a tabular input where each column corresponds to an independent scalar feature. Applying these methods to a sequence model therefore requires deliberate desing dicisions about how features are defined, how perturbations are applied and how attributions over the temporal dimesnion are reduced to a per-sensor importance score. Unless stated otherwise, the full temporal extent of a sensor channel is treated as a single unit, perturbations are applied uniformly across all 70 time steps of a given sensor simultaneously and the resulting attributions are aggregated over time to yield a scalar importance score per sensor. This choice reflects the physically motivated view that a sensor's contribution to the RUL prediction should be attributed to the sensor as a whole rather than decomposed into individual time steps, which would result in a 910-dimensional attribution space that is difficutlt to interpret and compare across methods.

Individual Conditional Expectation (ICE): For each of the 13 sensors, a uniform grid of 20 values is constructed between the 5th and 95th empirical percentile of that sensor's observed values across the dataset. The percentile bound are used in place of the global minimum and maximum to avoid evaluatiing the model in the tails of the distribution, where training data is sparse and prediction may be unreliable. For each grid value g and each instance in the dataset,

the entire time series of sensor j is replaced by the constant value g, that is, $x_{j,t} = g$ for all $t \in [1, 70]$, while all remaining sensor channels are kept at their observed values. This produces one ICE curve per instance, tracing how the predicted RUL changes as sensor j is swept across its range in isolation. The PDP is then computed as the pointwise mean of all ICE curves. Feature importance is derived as the range of the PDP curve, defined as the difference between its maximum and minmimum values, which captures the total variation in average predicted RUL attributable to sensor j.

Feature Ablation: The implementation follows the grouped ablatioin scheme provided by Captums's *FeatureAblation*, in which all 70 timesteps belonging to a given sensor channel are masked simultaneously as a single group rather than independently. This sensor-grouped formulation is essential for the present setting: ablating individual timesteps in isolation would produce attributions across 910 independent input positions, making cross-method comparison difficult and reducing the physical interpretability of the results. When sensor j is ablated, its entire time series is replaced by the training set mean computed independently for each channel, representing the expected signal in the absence of informative variation. Attributions are defined as the absolute change in prediction relative to the unpertrubed output, and global feature importance is obtained by averaging these absolute differences over all instances in the evaluation set.

Shapley Additive Explanations: To maintain consistency with the grouped ablation scheme described above, the 13 sensor channels are treated as 13 players in the cooperative game rather than operating on the 910-dimensional flattened input. Exact Shapley values are computed by enumerating all $2^{13} = 8192$ possible sensor coalitions and evaluating the model on each, which is computatinoally tractable for this group size. When a sensor group is absent from a coalition, its timesteps are replaced by a mean valued baseline as in the ablation method. For each coalitioin S, the masked input is constructed by replacing the features of all sensors not in S with the baseline and the model is queried to obtain $v(S)$. The Shapley values are then computed from these chached coalition outputs, yielding a per-sensor attribution for each instance in the dataset. A visualistation of the Shapley values is provided in the results chapter and implemented using the *shap* library.

Local Interpretable Model-Agnostic Explanations: The implementation uses Captum's `LIME` with a Lasso linear surrogate as the interpretable model class. Since LIME operates by perturbing the input in an interpretable feature space and fitting a weighted linear regression, the (13 x 70) input tensor is first flattened to a 910-dimensional vector. For each instance to be explained, 4000 perturbed samples are drawn in the neighbourhood of the input, each evaluated by the model. A proximity-weighted Lasso regression($\alpha = 0.01$) is then iftted to approximate the model locally, with the regularisation term enforcing sparsity in the resulting attribution vector. The zero vector is used as the reference baseline. After fitting, the 910-dimesninal weight vector is reshaped back to (13 x 70) and the time dimensioin is averaged to obtain sensor-level importance scores. The Lasso regularisation parameter α was selected to balance sparsity with attribution stability across instances.

Counterfactual Explanations: Counterfactual explanations are generated using the `DiCE` library, which implements a gradient -based search for minmally modified inputs that shift the

model's prediction into a specified target range. As `DiCE` requires a tabular input representation, the (13 x 70) tensor is flattened to a 910-dimensional feature vector before being passed to the `DiCE` data interface, which is initialised using the training set to define plausible feature ranges. For each query instance, the desired output range is specified as a window around the original prediction, so that the counterfactual represents a plasuible alternative trajectory with a meaningfully differen predicted RUL. The distance between the original and counterfactual input is pnealised using an L_1 norm, which encourages sparse modifications, that is, changes concentrated in a small number of features. Feature importance is subseqently derived by computing the magnitued of change per sensor, aggregated across the 70 timesteps, which identifies which sensors were most frequently or substantially modified to achieve the target prediction shift.

Permutation Feature Importance: For each sensor channel, its values are permuted randomly across all N samples in the evaluation set, while all other channels remain at their original values. This destroys the statistical association between that sensor and the target RUL without altering the marginal distribution of the sensor's values, so any resulting drop in predictive performance can be attributed to the loss of that sensor's information. Performance is measured as MSE on the normaliised RUL scale. To account for variablity in the permutation, the procedure is repeated with 5 independent random permutations and the mean and standard deviation of the MSE increase are reported per sensor. The use of 5 repeats provides a sufficiently stable estimate for 13 sensors while keeping the total number of model evaluation manageable.

Accumulated Local Effects: The domain of each sensor is partitioned into 20 bins using quantile-based bin edges, so that each bin contains approximately the same number of samples. Quantile-based binning is preferred over unifrom binning because sensor distributions may be skewed after operating-condition-specific normalisation and equal-frequency bins ensure that local effect estimates are based on suffiecient data throughout the feature range. Bin membership for each sample is determined by the mean value of that sensor's time series, which reduces the (N x L) sensor values to a scalar per sample for assignment purposes. Within each bin, the local prediction difference is computed by replacing the entire time series of senso j with the lower and uper bin boundary values in turm, and recording the resulting change in model output. These local differences are averaged within each bin and then accumulated across bins following 3.33. The accumulated effect curve is centered to zero mean over the empirical distribution of sensor values. A scalar importance scoe is derived as the range of the centered ALE curve, reflecting the total span of predicted RUL variation attributable to that senos under the local conditional distribution.

4.5 Unified Feature Importance Representation

Because the eight feature importance methods produce attributions in different units and on different scales, a unified representation is required before any cross method comparison can be made. For each method, the raw feature-level attributions are reduced to a single importance score per sensro by aggregating over the temporal dimension where applicable. The resulting

scores are then normalised using L_1 normalisation:

$$\mathrm{FI}_{\mathrm{L}_1} = \frac{|\mathrm{FI}_j|}{\sum_{k=1}^{13} |\mathrm{FI}_k|}, \tag{4.5}$$

so that the importance scores for all features sum to one for any given instance or aggregate. This normaliation ensures that the magnitude of attributions is comparable across methods regardless of their original scale, enabling direct visual and quantitative comparison in the results chapter.

5 Results

5.1 Verification of FI algorithms

section title: Algorithmic Verification of Feature Importance Methods

keep it general Feature importance algo & model

5.2 Comparison: Single vs Combined Models

global comparison

sensor comparison

time comparison

5.3 Feature Importance Results per Method

per method analysis

5.4 Comparison

correlation matrices feature importance of all methods, summed together -> heat maps

time component

5.5 Discussion

6 Conclusion and Outlook

6.1 Summary of Contributions

6.2 Answering the Research Questions

6.3 Outlook

Bibliography

Aas, Kjersti, Jullum, Martin and Løland, Anders (2021): *Explaining individual predictions when features are dependent: More accurate approximations to Shapley values*. Artificial Intelligence, Bd. 298, S. 103502. ISSN 00043702.

Apley, Daniel W. and Zhu, Jingyu (2019): *Visualizing the Effects of Predictor Variables in Black Box Supervised Learning Models*.

Breiman, Leo (2001): *Random Forests*. Mach. Learn., Bd. 45, S. 5–32. ISSN 0885-6125.

Fisher, Aaron, Rudin, Cynthia and Dominici, Francesca (2019): *All Models are Wrong, but Many are Useful: Learning a Variable's Importance by Studying an Entire Class of Prediction Models Simultaneously*.

Goldstein, Alex, Kapelner, Adam, Bleich, Justin and Pitkin, Emil (2015): *Peeking Inside the Black Box: Visualizing Statistical Learning With Plots of Individual Conditional Expectation*. Journal of Computational and Graphical Statistics, Bd. 24, S. 44–65. ISSN 1061-8600, 1537-2715.

Karimi, Amir-Hossein, Barthe, Gilles, Balle, Borja and Valera, Isabel (2020): *Model-Agnostic Counterfactual Explanations for Consequential Decisions*.

Kusner, Matt J., Loftus, Joshua R., Russell, Chris and Silva, Ricardo (2018): *Counterfactual Fairness*.

Lundberg, Scott and Lee, Su-In (2017): *A Unified Approach to Interpreting Model Predictions*.

Lundberg, Scott M., Erion, Gabriel G. and Lee, Su-In (2019): *Consistent Individualized Feature Attribution for Tree Ensembles*.

Molnar, Christoph (2022): *Interpretable Machine Learning*. Leanpub. Available online.

Ribeiro, Marco Tulio, Singh, Sameer and Guestrin, Carlos (2016): *Model-Agnostic Interpretability of Machine Learning*.

Shapley, L. S. (1953): *Stochastic Games**. Proceedings of the National Academy of Sciences, Bd. 39, S. 1095–1100.

Sundararajan, Mukund, Taly, Ankur and Yan, Qiqi (2017): *Axiomatic Attribution for Deep Networks*.

Verma, Sahil, Boonsanong, Varich, Hoang, Minh, Hines, Keegan E., Dickerson, John P. and Shah, Chirag (2022): *Counterfactual Explanations and Algorithmic Recourses for Machine Learning: A Review*.

Wachter, Sandra, Mittelstadt, Brent and Russell, Chris (2018): *Counterfactual Explanations without Opening the Black Box: Automated Decisions and the GDPR*.

Zeiler, Matthew D. and Fergus, Rob (2014): *Visualizing and Understanding Convolutional Networks*. In: *Proceedings of the European Conference on Computer Vision (ECCV)*, S. 818–833. Springer.

Štrumbelj, Erik and Kononenko, Igor (2014): *Explaining prediction models and individual predictions with feature contributions*. Knowledge and Information Systems, Bd. 41, S. 647–665. ISSN 0219-1377, 0219-3116.

Appendix

A.1 Anhang 1

Table A.1: Simulationsergebnisse, die den Lesefluss stören.

	$w = 8$			$w = 16$		
	$t = 0$	$t = 1$	$t = 2$	$t = 0$	$t = 1$	$t = 2$
$dir = 1$						
c	0,0790	0,1692	0,2945	0,3670	0,7187	3,1815
c	-0,8651	50,0476	5,9384	-9,0714	297,0923	46,2143
c	124,2756	-50,9612	-14,2721	128,2265	-630,5455	-381,0930
$dir = 0$						
c	0,0357	1,2473	0,2119	0,3593	-0,2755	2,1764
c	-17,9048	-37,1111	8,8591	-30,7381	-9,5952	-3,0000
c	105,5518	232,1160	-94,7351	100,2497	141,2778	-259,7326

A.2 Anhang 2

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipi-

scing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.